

Vladan Simic - Test Task 2 API Automation

Media Buyers API Contract | PHP + Codeception solution

Field	Value
Document purpose	Written evaluation and delivery summary for the API automation assignment
API resource	Media Buyers
Contract base URL	{{BASE_URL}}/api
Automation stack	PHP 8.2+, Codeception 5, REST, PhpBrowser, Asserts, JSON Schema validator
Date	June 12, 2026
Status	GitHub-ready solution package prepared

Executive Summary

The solution treats the PDF contract as the source of truth and implements a GitHub-ready PHP/Codeception API test framework. Because the assignment explicitly says that no live API exists, the suite is designed around clean test code, reusable abstractions, environment-based configuration, JSON Schema validation, and CI readiness rather than local execution against a real service.

Delivery snapshot

The project includes 12 Cest test methods and parameterized examples that cover 17 contract scenarios across GET and POST, including positive flows, field validation, schema validation, duplicate mbld behavior, and the empty-list acceptance criterion.

Repository Deliverables

Area	Delivered item	Why it matters
Test code	tests/api/MediaBuyerCest.php	Maps each important acceptance criterion to readable Codeception tests.
HTTP boundary	MediaBuyerApi client wrapper	Keeps endpoint paths and required headers outside test bodies.
Payloads	MediaBuyerPayloadFactory	Builds valid requests and negative overrides without hard-coded JSON in tests.
Schemas	get-media-buyers-schema.json and post-media-buyer-schema.json	Validates successful responses against the supplied contract.
Assertions	MediaBuyerAssertions	Centralizes protocol, schema, response, and

		validation-error checks.
CI	.github/workflows/api-tests.yml	Shows how the suite plugs into GitHub Actions once API_BASE_URL exists.

Written Evaluation

1. Scenario selection

I selected scenarios that protect the resource contract and the highest-risk integration behavior. For GET, that means status, JSON content type, schema shape, array semantics, required fields, email syntax, active flag typing, and ID uniqueness. For POST, that means valid creation, server-generated ID, request-to-response field mapping, boolean-to-integer active conversion, required-field errors, boundary validation, invalid types, and duplicate mbld handling.

I intentionally left out authentication, authorization, filtering, pagination, rate limiting, and performance because the contract does not define them. Those would be added when the API contract expands rather than guessed in code.

2. Abstractions introduced

The suite introduces a small HTTP client wrapper, payload factory, JSON Schema assertion helper, validation-error assertion helper, and fixture area. At 8 tests this keeps the code clean; at 80 tests it prevents duplication, keeps endpoint changes localized, and makes new boundary cases cheap to add. Schemas remain separate artifacts so they can later be generated or compared against OpenAPI.

3. Contract drift detection

I would make the API contract versioned and canonical, ideally as OpenAPI plus generated JSON Schemas. CI should run an OpenAPI diff on every backend pull request, fail when response shapes or validation rules change without test updates, and require QA review on contract-changing PRs. Schema files in this repository should be regenerated or intentionally updated when the contract changes, creating a visible review point.

4. Tools, including AI

For daily QA ownership I would use Codeception for PHP API tests, GitHub Actions for CI, Allure or Codeception HTML/XML reports for visibility, OpenAPI diff tooling for drift, Prism or WireMock for mock-contract environments, and Pact-style consumer contract tests if multiple services consume the API. AI tools are useful for drafting boundary matrices, generating first-pass test cases from an OpenAPI document, reviewing duplicated assertions, and summarizing flaky-test trends, but final scenario selection and risk decisions should remain human-owned.

5. Challenging automation situation

A challenging API automation situation I would highlight is test instability caused by shared state and asynchronous backend processing. The fix is to isolate data with unique builders, make setup and teardown explicit, replace fixed sleeps with polling helpers and clear timeouts, separate contract tests from longer end-to-end workflows, and improve failure diagnostics by logging request IDs, payloads, response bodies, and environment metadata. That approach turns flaky tests into actionable feedback instead of noise.

Acceptance Criteria Traceability

GET /api/mediabuyers

ID	Contract criterion	Automated coverage
G1	Returns HTTP 200 with Content-Type application/json.	listMediaBuyersReturnsJsonArrayMatchingSchema
G2	Response matches get-media-buyers-schema.json.	listMediaBuyersReturnsJsonArrayMatchingSchema
G3	data is always an array, including empty state.	listMediaBuyersReturnsJsonArrayMatchingSchema; emptyMediaBuyerListStillReturnsDataArray
G4	Each item includes id, mbId, initials, name, email, slackUserId, active.	seeEveryListedBuyerHasRequiredFields helper
G5	email values are syntactically valid.	listedMediaBuyersRespectFieldLevelContract
G6	active is integer 0 or 1, not boolean/string.	listedMediaBuyersRespectFieldLevelContract
G7	id values are unique across response.	listedMediaBuyerIdsAreUnique

POST /api/mediabuyers

ID	Contract criterion	Automated coverage
P1	Valid request returns 200, JSON, and post schema.	createMediaBuyerMapsBooleanActiveToInteger
P2	data.id is server-generated positive integer.	seeServerGeneratedPositiveId helper
P3	Created buyer echoes mbId, initials, name, email, slackUserId.	seeCreatedBuyerMatchesPayload helper
P4	active true -> 1 and false -> 0.	Parameterized examples for true and false
P5	Missing required fields return 400 and name missing field.	requiredFieldsAreValidated examples
P6	not-an-email returns 400 and mentions invalid email.	invalidEmailsRejected
P7	initials longer than 2 returns exact message.	invalidInitialsLengthsRejected
P8	name shorter than 2 or longer than 30 returns length error.	invalidNameLengthsRejected examples
P9	mbId abc returns 400.	nonNumericMbIdsRejected

P10	active yes returns 400.	nonBooleanActivelsRejected
P11	Duplicate mbld fails on second request.	duplicateMbldIsRejectedOnSecondCreate

Assumptions and Open Questions

- API_BASE_URL includes the /api prefix; tests call /mediabuyers relative to that base.
- Content-Type may include a charset; tests assert application/json is present.
- Duplicate mbld should ideally be 409 Conflict, but the test accepts 400 or 409 because the contract leaves this open.
- The empty-list criterion requires a seeded or mock environment with zero media buyers.
- initials and slackUserId are optional in requests, but required in successful responses by the supplied schemas.

Suggested GitHub Submission Steps

1. Create a public GitHub repository named media-buyers-api-tests or similar.
2. Upload the contents of the project ZIP as the repository root.
3. Keep README.md visible at the root because it explains the suite design, assumptions, and future CI path.
4. Attach or link this Word document as the written evaluation deliverable.